Leitz Icon Windows SDK

Label Printer

ICON

Edition: 01/2016

Copyright
This documentation as well as translation
hereof is property of Esselte.

The replication, conversion, duplication or disclosures of
the whole manual or parts of it for other intentions than its
original intended purpose demands the written
authorization by Esselte.

Editor
Regarding questions or comments please contact:

Kris Vandermeulen
Esselte Business bvba
Industriepark Noord 29
9100 Sint Niklaas
Belgium
Tel.: +32 3760 33 04
e-mail: kvandermeulen@esselte.com

Topicality
Due to the constant further development of our products
discrepancies between documentation and product can
occur.

Table of Contents

1. HOW TO INCLUDE SDK INTO A PROJECT 4

2. SDK FUNCTIONALITY 5

2.1 ENTITIES 5

2.1.1 LILabel 5

i. Description 5

ii. Properties: 5

iii. Methods: 5

2.1.2 LILabelElement 6

i. Description 6

ii. Properties: 6

iii. Methods: 7

2.2 PRINTER CONTROLLER 7

2.2.1 LIPrinterController 7

i. Description 7

ii. Methods: 7

2.3 GLOBAL SETTINGS 8

2.3.1 LIGlobalSettings 8

i. Description 8

ii. Properties: 8

3. EXAMPLES OF USAGE 9

1. How to include SDK into a project

To deal with SDK the following dll files should be referenced in project:

- *LeitzIconLabelStudioCore.dll*

  This is the library with SDK core.

- *LeitzIconCommon.dll*

  This library contains base operations, interfaces, data contracts and specific controls.

- *LeitzIconDataProvider.dll*

  This library contains different metadata used in the SDK, for example XML label data etc.

- *PrintHelper.dll*

  This library is used for low-level printer operations like setting paper or custom paper.

SDK entities and classes are in the namespace *LeitzIconSDK*.

Files that should be placed into the directory with user's application:

- *LeitzIconPrinterStatusHost.exe*

  This file is used for reading the printer statuses.

- *LeitzIconPrinterStatusService.dll*

  This file is used for *LeitzIconPrinterStatusHost.exe* tool

2. SDK functionality

SDK covers further functionality:

- open label
- get label elements, read their IDs and values, update element value
- save label
- get printers list
- get printer status
- get cartridge information
- get cartridge status
- Print a label

2.1 Entities

2.1.1 *LILabel*

i. Description

- Entity that represents label.

ii. Properties:

- *Id* - type of Label
- *Elements* - collection of label elements
- *FilePath* – path to the label

iii. Methods:

- ***Print (parameters: numberOfCopies[optional], addresses[optional])***
  - Prints the label.
  - If *numberOfCopies* is set than the label will be printed the defined number of times.
  - If *addresses* array is set than each array item will be printed on the corresponding place of the label.
- ***Save ()***
  - Saves changes into file.
- ***Open (parameters: filePath)***
  - Opens the label from file system.
- ***GetLabelImage ()***

- Returns rendered label as ImageSource (WPF image type).

- ***GetLabelBitMap (parameters: scaleX [optional], double scaleY [optional])***
  - Returns rendered image of the label as Bitmap.
  - Could be scaled by passing corresponding parameters.

2.1.2 LILabelElement

i. Description

- Entity that represents label element.

There are inherited classes for different types of elements:

- *LIAddressLabelElement*
- *LIBarcodeLabelElement*
- *LICircleLabelElement*
- *LICounterLabelElement*
- *LIDateLabelElement*
- *LIImageLabelElement*
- *LILineLabelElement*
- *LIRectangleLabelElement*
- *LITextLabelElement*

ii. Properties:

- ***LIAddressLabelElement***
  - Id (string)
  - Value (string)
- ***LIBarcodeLabelElement****:*
  - Id (string)
  - Value (string)
- ***LICircleLabelElement:***
  - Id (string)
- ***LICounterLabelElement:***
  - Id (string)
  - Value (unit)
- ***LIDateLabelElement:***
  - Id (string)

- Value (DateTime)

- ***LIImageLabelElement:***
  - Id (string)
  - Value (ImageSource)
- ***LILineLabelElement:***
  - Id (string)
- ***LIRectangleLabelElement:***
  - Id (string)
- ***LITextLabelElement**:*
  - Id (string)
  - Value (string)

iii. Methods:

- ***GetElementBitMap (parameters: scaleX [optional], double scaleY [optional])***
  - Returns image of *LIImageLabelElement* as BitMap.
- ***SetElementImage (string path)***
  - Sets the *LIImageLabelElement* image from file system's file.

2.2 Printer Controller

2.2.1 *LIPrinterController*

i. Description

- *Static class for dealing with printers.*

ii. Methods:

- ***GetPrinters()***
  - Returns list of Leitz Icon printers' names.
- ***SetActivePrinter (parameter: printerName)***
  - Method for setting current active printer.
  - Receives printer name as parameter.
- ***IsPrinterOnline()***
  - Returns true if printer is online, false otherwise.
- ***GetCartridgeStatus (parameters: force [optional])***

- Returns *CartridgeStatus* object which contains information about status of inserted cartridge.
- It has further fields: Total, Used and Free.
- If “force” parameter is true than printer status will be read immediately (cached data from previous request won’t be used).

- ***ToString()***
  - Returns combined info in 1 string.
- ***GetBatteryStatus (parameters: force [optional])***
  - Returns *BatteryState* enumeration (NoBat, BatFail, Charging, NoCharging, BatFull and Error).
  - If “force” parameter is true than printer status will be read immediately (cached data from previous request won’t be used).
- ***GetCartridgeInfo (parameters: force)***
  - Returns information about the inserted cartridge.
  - It is a string with cartridge type, size and color.
- ***Print (parameters: label, numberOfCopies[optional])***
  - Prints the label
  - If *numberOfCopies* is set than the label will be printed the defined number of times.

2.3 Global Settings

2.3.1 *LIGlobalSettings*

i. Description

- Static class for dealing with SDK global settings.

ii. Properties:

- ***MailerId***
  - USPS mailer id which used for IMBC barcode generation.
- ***PrintResolution***
  - Printing resolution, it is *DeviceResolution* object (it has x and y fields). X – Horizontal resolution (can be only 300 dpi), y – vertical resolution (can be 300 or 600 dpi).
- ***CutAfterEachPage***
  - Bool flag for setting of the cutting after each label.

3. Examples of usage

1. Opening the label:

```
LILabel label = LILabel.Open(filepath);
```

2. Rendering the label onto Windows Forms PictureBox control:

```
pictureBox.Image = label.GetLabelBitMap();
```

3. Getting of an array of printers

```
string[] printers = LIPrinterController.GetPrinters();
```

4. Setting of an active printer:

```
IPrinterController.SetActivePrinter(printerName);
```

5. Checking the printer availability:

```
LIPrinterController.IsPrinterOnline()
```

6. Getting the label element's value (LIAddressLabelElement in this case):

```
LILabelElement element = label.Elements[0];
if (element is LIAddressLabelElement)
       textValueTBox.Text = ((LIAddressLabelElement) element).Value;
```

7. Setting the label element's value (LIAddressLabelElement in this case):

```
((LIAddressLabelElement) element).Value = textValueTBox.Text;
```

8. Save the label:

```
label.Save();
```

9. Print the label

```
label.Print();
```

10. Getting of printer statuses:

```
string cartridge Info = LIPrinterController.GetCartridgeInfo(true).ToString();
string cartridgeStatus = LIPrinterController.GetCartridgeStatus(false).ToString();
string batteryStatus =
LIPrinterController.GeBatteryStatus(false).State.ToString();
string printerStatus = LIPrinterController.IsPrinterOnline() ? "Online" :
"Offline";
```

There are also Windows Forms and WPF SDK sample projects available for exploration and testing of SDK functionality.